AI7 · Inferencia y despliegue

Prefill · decode · KV-cache · batching · cuantización · memoria · coste

AI Surfer · N3-AI Engineering

Mapa del módulo

Pregunta	Pieza
¿Por qué tarda en empezar?	Prefill y TTFT
¿Por qué escribe a ese ritmo?	Decode y KV-cache
¿Cómo atiendo a muchos?	Batching continuo
¿Cómo lo hago caber?	Cuantización y memoria
¿Sale a cuenta?	Coste por millón de tokens

Dos fases, dos cuellos de botella

Fase	Limita	Métrica
Prefill (prompt)	Cálculo	**TTFT**
Decode (token a token)	**Memoria** (leer pesos y caché)	tokens/s

Mide siempre **p50 y p95**, con **tu** tráfico.

KV-cache: qué ahorra

Paso *t*, atención:

- Sin caché: reprocesas *t* tokens → **O(t²)** por paso, **O(T³)** total
- Con caché: 1 token nuevo mira *t* claves → **O(t)** por paso, **O(T²)** total

Ratio ≈ (T+2)/3: **11× con T=32 · 171× con T=512**

Comprueba primero que la salida es **idéntica**.

KV-cache: lo que cuesta

`2 × capas × cabezas_KV × head_dim × bytes` por token

Modelo ~8 B con GQA, fp16: **128 KiB/token**

→ 32k tokens ≈ **4 GiB por petición**

Sin GQA: **4× más**. La caché, no los pesos, limita la concurrencia.

Batching: el compromiso

- Lote mayor → **más tokens/s totales**
- Lote mayor → **menos tokens/s por usuario**

Una lectura de pesos sirve a *B* peticiones.

Throughput = coste · *tokens/s por usuario* = experiencia

Continuous batching

	Estático	Continuo
Admite peticiones nuevas	Al acabar **todo** el lote	En **cada paso**
Huecos vacíos	Muchos	Casi ninguno
TTFT	Alto	Bajo

En la simulación: ~2× tokens por paso y TTFT medio a una fracción.

PagedAttention: KV-cache en bloques, sin fragmentación.

Cuantización: la idea

```
escala = max|w| / qmax          (127 en int8 · 7 en int4)
entero = redondear(w / escala)
w ≈ entero × escala
```

fp32 → int8: **4× menos** · 4 bits por grupos: **~7× menos** que fp32

Granularidad

Esquema	Una escala por…	Con un canal atípico
Global	toda la matriz	~14 % de error en el resto
Canal	cada fila	~0,5 %
Grupo (32–128)	cada trozo	lo confina al grupo

Los atípicos mandan: AWQ, SmoothQuant, LLM.int8.

Mide la calidad, no solo el tamaño

- int8: casi gratis · int4: aguanta · 3 y 2 bits: precipicio
- La **KL** frente al original avisa antes que el acierto
- Con un modelo real: **tu eval del AI4**, tus casos difíciles

Toda optimización de inferencia es un **cambio de modelo**.

Presupuesto de memoria

```
GPU ≈ pesos + KV-cache + activaciones
```

8 B con GQA, contexto 32k, GPU de 24 GB:

- fp16/fp16: **1** petición
- pesos 4 bits: **4**
- - KV-cache a 8 bits: **8**

Palancas: GQA · KV a 8 bits · pesos · límites · más GPU

Servir en producción

- vLLM (estándar en GPU) · SGLang · TensorRT-LLM · llama.cpp/Ollama · servicios gestionados
- Límites de concurrencia y longitud, *timeouts*, *streaming*, `/health`
- **Prueba de carga:** peticiones/s frente a p95 para varias concurrencias
- Sin servidores colgados: `TestClient`, `httpx` + ASGI

Comprueba siempre la versión.

Paralelismo no es lote

Más peticiones simultáneas **sin agruparlas** no multiplican el rendimiento: compiten por el mismo hardware.

El *batching* vive **dentro del motor**.

Coste por millón de tokens

```
€/M = coste_hora / (tokens_s × 3600 × utilización) × 10⁶
```

2 €/h · 1500 tok/s: **0,37 €/M al 100 % → 2,5 €/M al 15 %**

Falta sumar: redundancia, guardias, ingeniería.

¿Servir tú o usar una API?

Compensa: volumen alto y constante, privacidad, modelo propio, latencia específica.
No compensa: tráfico bajo o irregular, equipo pequeño, calidad de frontera.

Decide con **tus** medidas y **tus** supuestos explícitos.

Entregable

Informe de despliegue: TTFT/tokens/s/p95 · cuantización con calidad medida · presupuesto de memoria · tabla de coste y **recomendación**

Es tu pieza de portafolio de inferencia